Key Exchange

CS 161 Spring 2026 - Lecture 12

Announcements

- Feedback: `henrycg.com/feedback`
- Thanks to those of you who sent in feedback!

Last time: Authenticated Encryption

- Chosen ciphertext attack (CCA)
  - Attacker can see ENCRYPTIONS of messages of its choosing
  - Attacker can see DECRYPTIONS of ciphertexts of its choosing

- Authenticated encryption: Guarantees confidentiality and integrity
  - Also CCA secure

- Can build AE from CPA-secure encryption and a secure MAC:
  - Encrypt-then-MAC: $\text{Enc}(K_1, M) \,||\, \text{MAC}(K_2, \text{Enc}(K_1, M))$
  - Other variations are dangerous

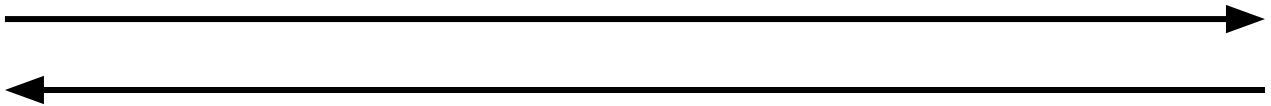
Cryptography roadmap

	Secret-key crypto	Public-key crypto
Encryption	✅ One-time pad ✅ CPA security ✅ Counter-mode encryption ✅ CBC encryption ✅ Authenticated encryption	– Key exchange (e.g., Diffie-Hellman) – Public-key encryption (e.g., ElGamal)
Authentication	✅ Message auth. codes	– Digital signatures (e.g. RSA sigs)
Building blocks	✅ Pseudorandom functions ✅ Block ciphers (e.g., AES) ✅ Hash functions (e.g., SHA2)	– Discrete-log problem

The first 3000 years of cryptography: Secret-key crypto









Key *K*

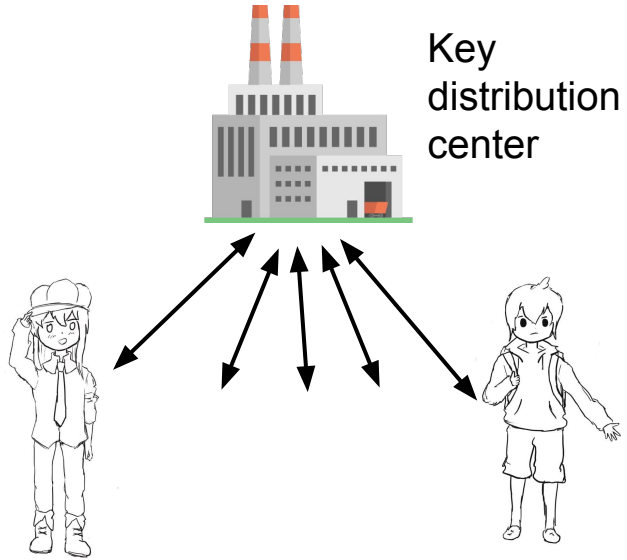
Key *K*

Sharing secrets is inconvenient

- Especially on a computer network with thousands of devices
- Especially when people add devices all the time
- How do two new devices establish a shared secret?

Bad solution: Key distribution center (KDC)

- Alice and Bob each share a secret with KDC
- KDC can set up shared Alice-Bob secret
- Kerberos works like this



Problem: Centralized

- What if KDC fails?
- What if attacker compromises KDC?
- :-(

In 1974, Ralph Merkle asked:

Can Alice and Bob agree on a shared secret key by communicating over a public network?



Key K

Key K

Project 2 looks more reasonable, maybe because your description Project 1 is muddled terribly. Talk to me about these today.

C.S. 244
FALL 1974

Ralph Merkle

Project Proposal

Topic: Establishing secure communications between seperate secure sites over insecure communication lines.

Assumptions: No prior arrangements have been made between the two sites, and it is assumed that any information known at either site is known to the enemy. The sites, however, are now secure, and any new information will not be divulged.

Method 1: Guessing. Both sites guess at keywords. These guesses are one-way encrypted, and transmitted to the other site. If both sites should chance to guess at the same keyword, this fact will be discovered when the encrypted versions are compared, and this keyword will then be used to establish a communications link.

Discussion: No, I am not joking. If the keyword space is of size

(Unauthenticated)

Key exchange

Adversary sees all messages between Alice and Bob

(Assume for now adv doesn't tamper with them)





pk_A

pk_B



Key K

Key K

(Unauthenticated)

Key exchange: Definition

An (unauthenticated) **key-exchange scheme** consists of two efficient algorithms:

- (sk, pk) ← KeyGen() // Generate each party's key pair
- K ← SharedSecret(sk, pk) // Derive shared secret from keys

Correctness: Alice and Bob agree on the same secret

For all (sk_A, pk_A), (sk_B, pk_B) output by KeyGen(): SharedSecret(sk_A, pk_B) = SharedSecret(sk_B, pk_A).

Security: It's infeasible for an eavesdropper to recover the shared secret.

For all (sk_A, pk_A), (sk_B, pk_B) output by KeyGen(), for all efficient adversaries *Adv*:

Pr[*Adv*(pk_A, pk_B) = SharedSecret(sk_A, pk_B)] < "negligible."

How can key exchange be possible?

- Attacker sees everything that Alice and Bob see.
- Bob and attacker have **the same information** about Alice's secret

  ⇒ Anything that Bob knows about Alice's secret, attacker also knows!

- Impossible?

Merkle's elegant key-exchange scheme ("Merkle puzzles")

Alice and Bob: Agree on shared secret in time N

Eavesdropper: Must run in time $\geq N^2$ to recover secret

If $N = 2^{35}$, this gap is significant:

- Alice and Bob = 30 seconds on a laptop
- Eavesdropper = $1 million of compute

Merkle's elegant key-exchange scheme ("Merkle puzzles")

Uses: A public hash function $H : \{1, \ldots, N^2\} \rightarrow \{0,1\}^{256}$

Alice's sk_A
Random $a_1, \ldots, a_N \in \{1, \ldots, N^2\}$

Bob's sk_B
Random $b_1, \ldots, b_N \in \{1, \ldots, N^2\}$



$pk_A =$

$H(a_1)$
$H(a_2)$
…
$H(a_N)$

Alice and Bob run in time N

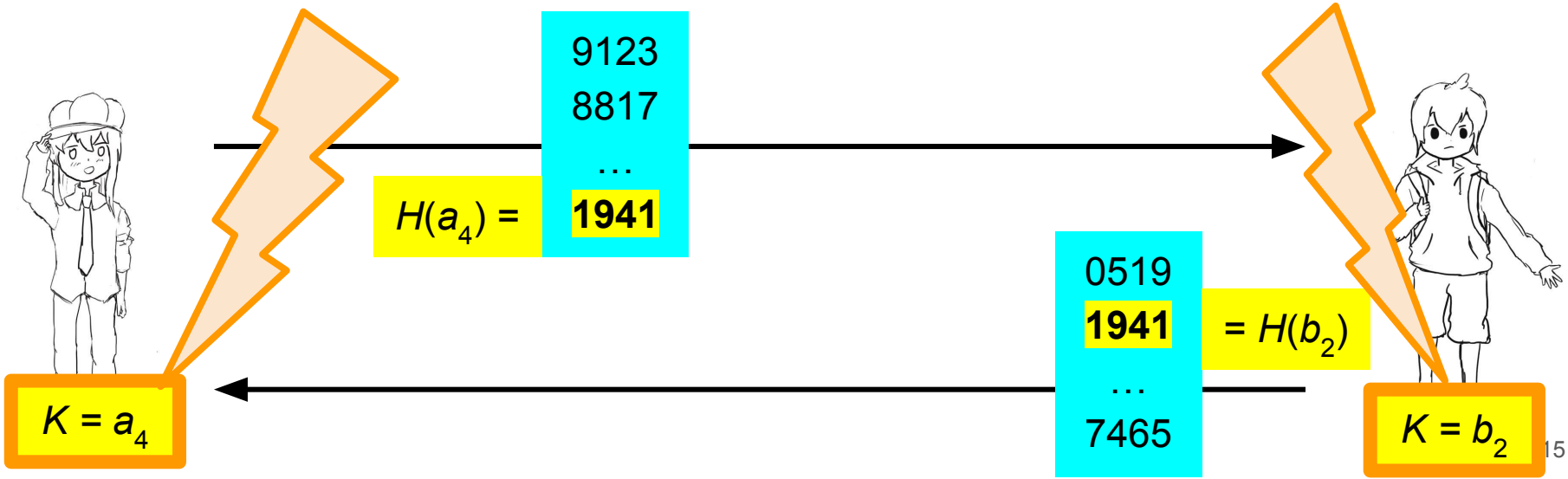
$H(b_1)$
$H(b_2)$
…
$H(b_N)$

$= pk_B$

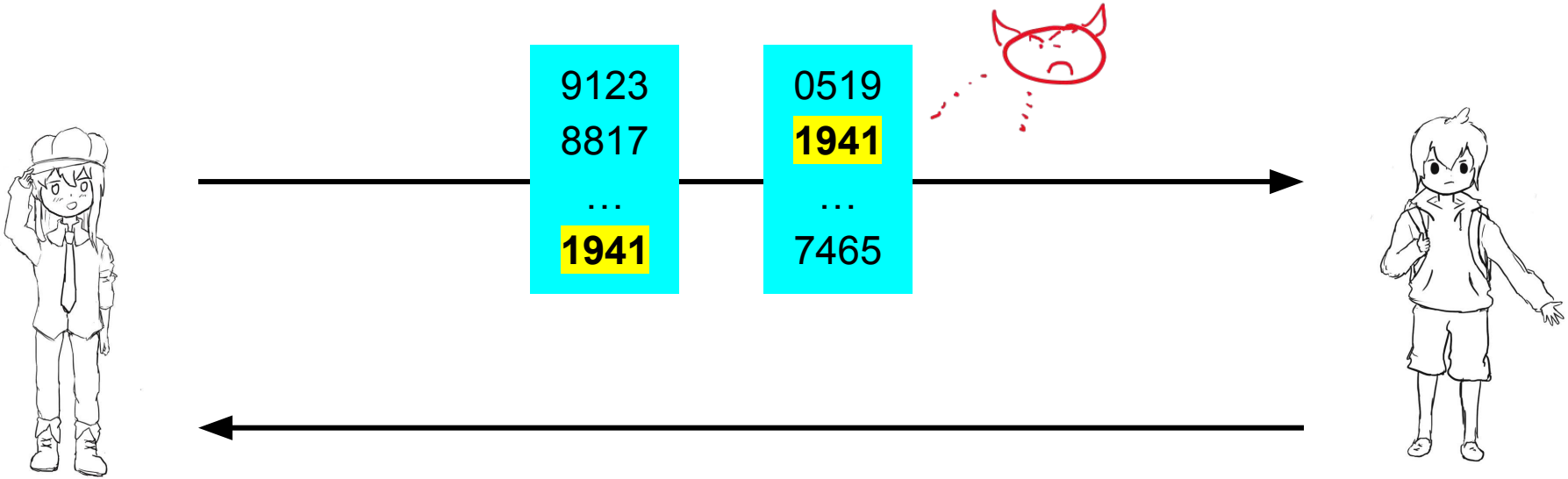


By a variant of the Birthday Paradox:

Alice and Bob's lists will likely share ≥ 1 hash in common



Merkle's key exchange: Security

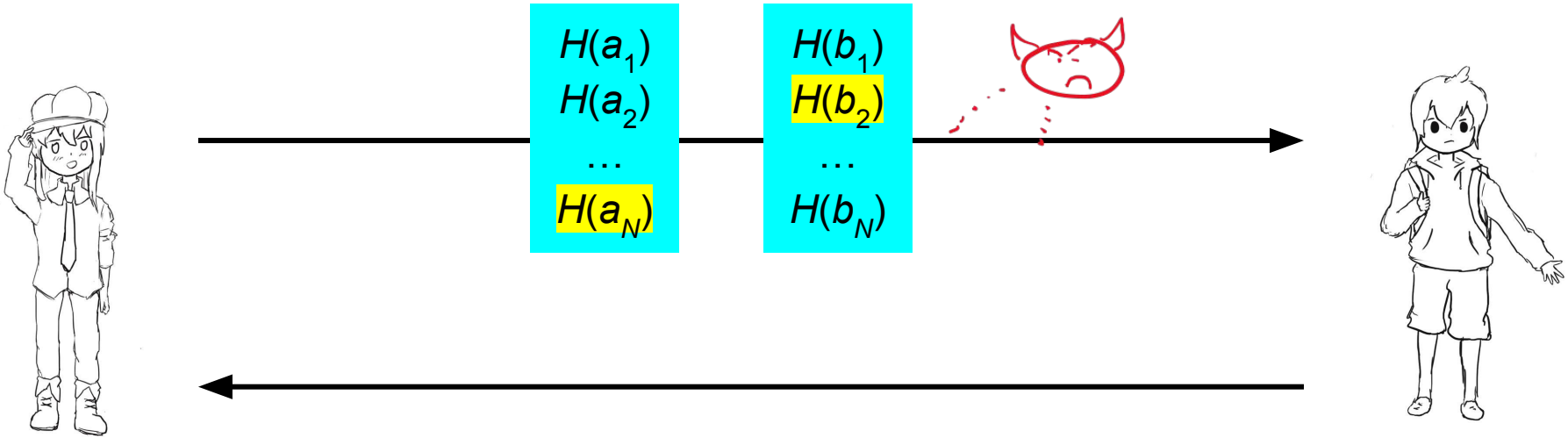


Merkle's puzzles: Security idea

Attacker gets HASH of Alice and Bob's shared secret

→ Attacker must brute-force guess secret: H(1), H(2), H(3), H(4), …

→ There are N^2 possible values to guess ⇒ **Attack takes ≥ N^2 time**



Merkle puzzles

Alice and Bob: Agree on shared secret in time N

Eavesdropper: Must run in time $\geq N^2$ to recover secret



In his class project:

- Merkle invented the concept of public-key cryptography
- Gave the first public-key cryptosystem

Project 2 looks more reasonable, maybe because your description Project 1 is muddled terribly. Talk to me about these today.

C.S. 244
FALL 1974

Ralph Merkle

Project Proposal

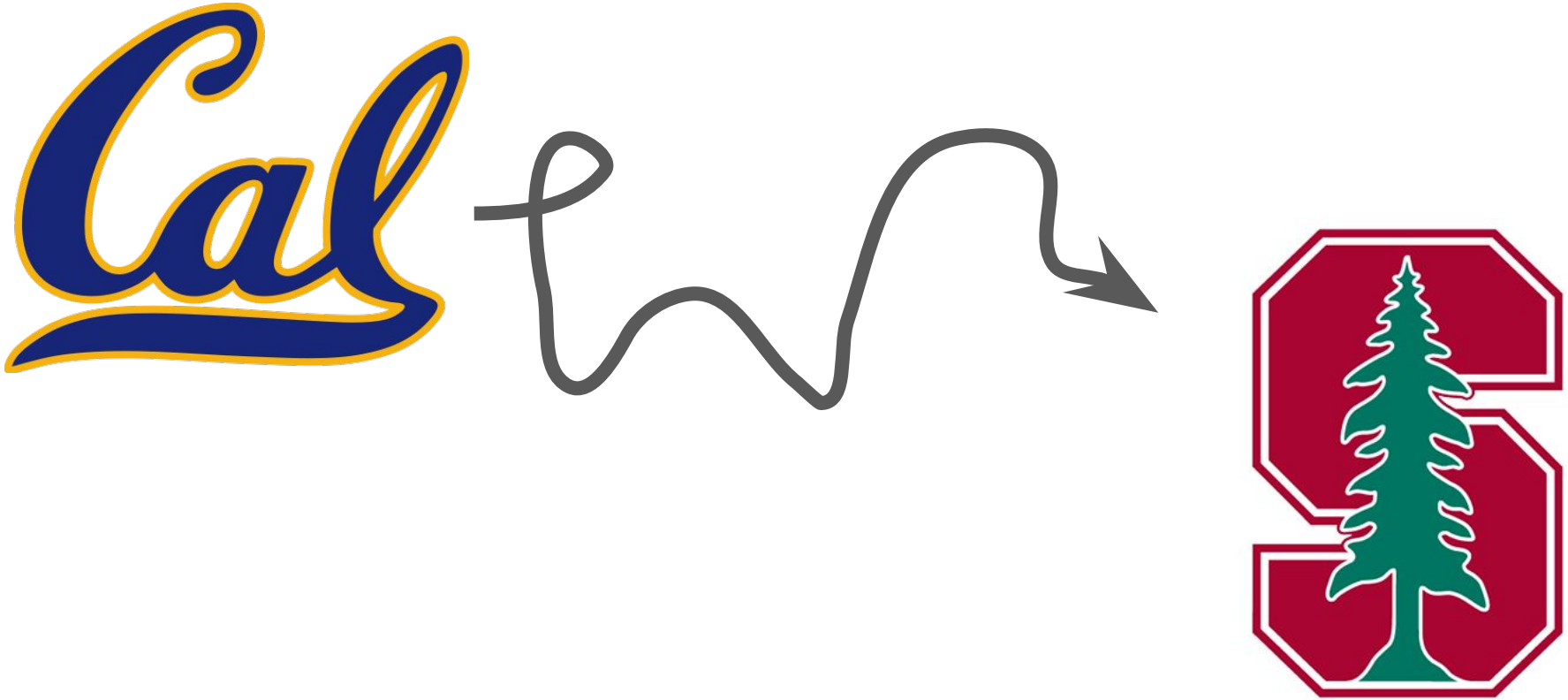
Topic: Establishing secure communications between seperate secure sites over insecure communication lines.

Assumptions: No prior arrangements have been made between the two sites, and it is assumed that any information known at either site is known to the enemy. The sites, however, are now secure, and any new information will not be divulged.

Method 1: Guessing. Both sites guess at keywords. These guesses are one-way encrypted, and transmitted to the other site. If both sites should chance to guess at the same keyword, this fact will be discovered when the encrypted versions are compared, and this keyword will then be used to establish a communications link.

Discussion: No, I am not joking. If the keyword space is of size

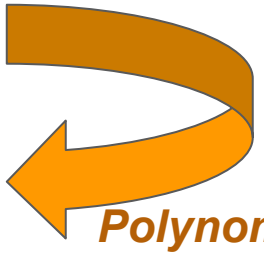
And then…



Merkle puzzles

Alice and Bob: Agree on shared secret in time N

Eavesdropper: Must run in time $\geq N^2$ to recover secret



Polynomial gap

Diffie-Hellman Key Exchange (1976):

Alice and Bob: Agree on shared secret in time poly(N)

Eavesdropper: Must run in time $\geq 2^{N/2}$ to recover secret

Exponential gap

Diffie-Hellman Key Exchange

Textbook Chapter 10

Public-key cryptography requires new math

- Merkle puzzles just use a cryptographic hash function
- We don’t know how to build exponentially secure key ex. from hash functions
  - One of the great open problems in cryptography
- Public-key crypto relies on the hardness of more-“structured” problems
- Examples: Factoring, the discrete-log problem, …
  - A big-enough quantum computer breaks many of these problems in **quantum** polynomial time
  - Latest generation of schemes are not known to be vulnerable to quantum attack
  - But they are more complicated and more costly :-(

Diffie-Hellman Key Exchange

Public parameters: – An n-bit prime p // "modulus"

– A number g in $\{0, \ldots, p - 1\}$ // "generator"

. . . **Carefully chosen to have certain properties!**

KeyGen() → (sk, pk):

– Pick $a \leftarrow_R \{0, \ldots, p - 1\}$

– Return (sk, pk) ← $(a, g^a \bmod p)$ // Can be computed efficiently

SharedSecret(sk = a, pk = B) → K:

– Return $K \leftarrow B^a \bmod p$.

Diffie-Hellman key exchange

Public parameters: – An n-bit prime p // "modulus," often 2048 bits long

– A number g in {0, …, p - 1} // "generator"

Alice's sk_A
Random $a \in \{0, \ldots, p\text{-}1\}$

Bob's sk_B
Random $b \in \{0, \ldots, p - 1\}$



$pk_A = g^a \bmod p$

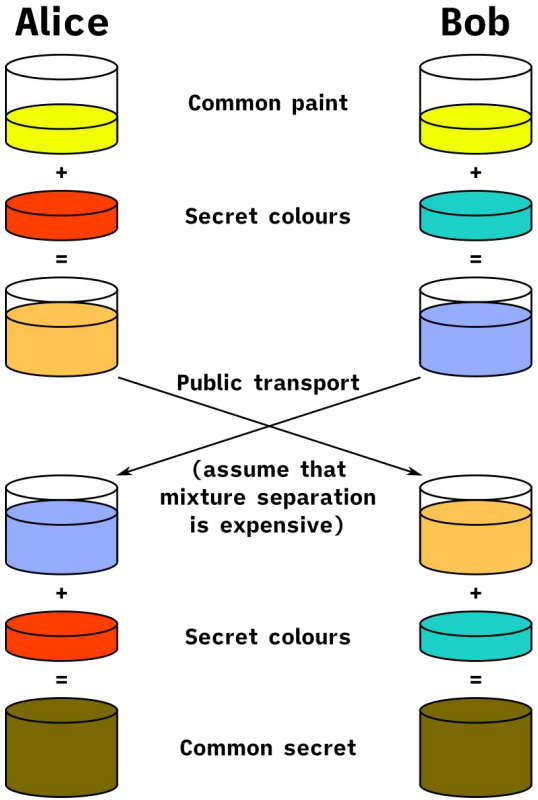


$pk_B = g^b \bmod p$

$K = (pk_B)^a = (g^b)^a = g^{ab}$

$K = (pk_A)^b = (g^a)^b = g^{ab}$

Diffie-Hellman key exchange as paint mixing





Diffie-Hellman key exchange: Properties

Correctness: Alice and Bob agree on the same secret

- Holds because: $(g^b)^a = g^{ba} = g^{ab} = (g^a)^b \bmod p$

Security: It's infeasible for an eavesdropper to recover the shared secret.

- Attacker's job:
  - **Given** $(p, g, g^a \bmod p, g^b \bmod p)$
  - **Compute:** $g^{ab} \bmod p$
- How hard is this problem? We don't know!
- We assume that this "computational Diffie-Hellman problem" (CDH) is hard
  - DH key exchange is secure under this assumption
- Related to the "discrete-log problem"

How hard is the computational Diffie-Hellman problem?

If the modulus is an *n*-bit prime…

- For the version we saw, best attack: **~ $2^{n^{\{1/3\}}}$ time** = "sub-exponential time"
  ⇒ Need 3074-bit keys to ensure that best attack takes time 2^{128}

- For the fancier "elliptic-curve" version, best attack:
  ~ $2^{n/2}$ time = exponential time
  ⇒ Need 256-bit keys to ensure that best attack takes time 2^{128}

- With a quantum computer, best attack:
  ~ n^2 time = polynomial time (**easy!**)

Discrete-log problem

Public parameters: – An n-bit prime p // "modulus"

– A number g in {0, …, p - 1} // "generator"

…**Carefully chosen to have certain properties!**

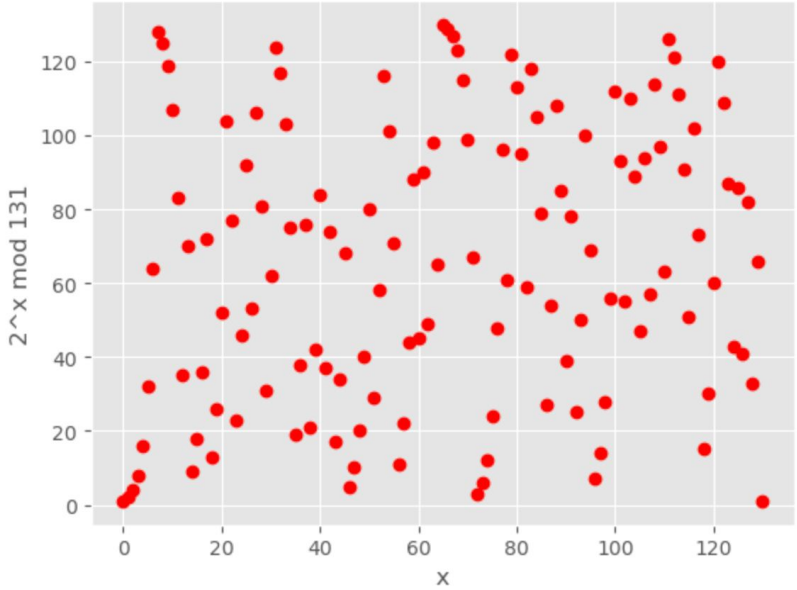
Discrete-Log Problem

Given: $g^a \bmod p$, for random a in {0, …, p - 1}

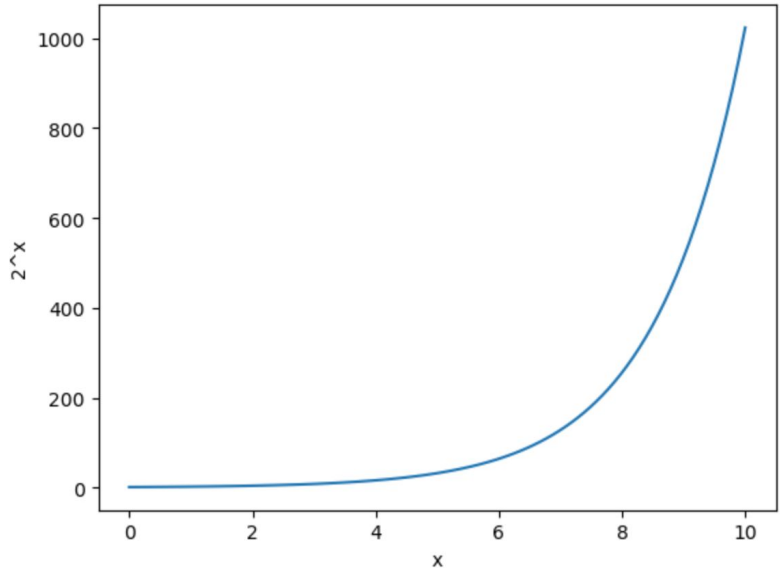
Find: a

→ For DH key exchange to be secure, it is necessary (not sufficient) that the discrete-log problem is hard.

Discrete Log Problem

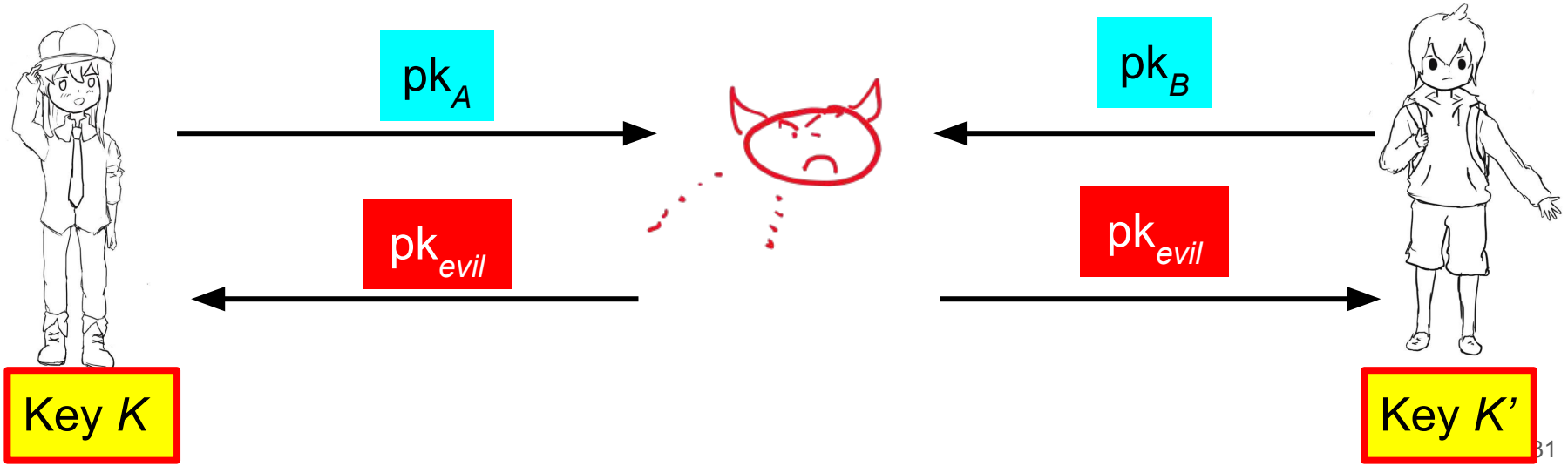


Plot of $g^a \bmod p$



Plot of g^a

Unauthenticated key exchange does not protect against active attacks!



Summary: Diffie-Hellman Key Exchange

- Key exchange: Agree on a shared secret over a public channel
  - Merkle puzzles = polynomially secure
  - Diffie-Hellman = exponentially secure (in the right setting)

- Diffie-Hellman algorithm:
  - Alice chooses a. Sends $g^a \bmod p$ to Bob
  - Bob chooses b. Sends $g^b \bmod p$ to Alice
  - Shared secret is $(g^a)^b = (g^b)^a = g^{ab} \bmod p$

- Unauth. key exchange does not protect against against active attacks

Next time: Public-key encryption